

#UNIT-03

#Search Engines -

(i) An automated system designed to discover, index, and rank the vast ocean of web content

(ii) Helps users effectively find relevant information from the internet

(iii) Pillars:

- Crawling = discovering & fetching web content
- Indexing = organizing and storing this content in a searchable format
- Ranking = determining the relevance & order of search results

(iv) Architecture:

- Web crawler = discovers & retrieves web pages
- Indexing engine = processes & stores content into searchable data structures
- Storage & caching = persistent storage for indexed data & quick retrieval mechanisms
- Query Processor = analyzes user queries & prepares them for ranking
- Ranking engine = determines relevance of results

#Web Crawling -

(i) Purpose: discover & fetch web content for indexing

(ii) Starts with seed URLs → follows links → extracts data (HTML, metadata)

(iii) Key aspects

- URL prioritization
- Distributed crawling
- Freshness management
- Rate limiting

(iv) Pipeline

- Crawling = fetch content (bots, HTTP)
- Indexing = organize data (B-trees, inverted ~~tree~~ index)
- Query processing = interpret queries (NLP, parsing)
- Ranking = order results (TF-IDF, ML models)
- Caching = speed up retrieval (Redis, Memcached)

Indexing -

- (i) Organizes crawled data into searchable format
- (ii) Maps Keywords $\rightarrow$ documents for fast retrieval
- (iii) Core structure = Inverted index (term $\rightarrow$ list of docs)
- (iv) Enables fast lookup ($O(1)$ / $O(\log n)$)
- (v) Index data structures
 - B-trees = efficient for ordered/range queries
 - Trie (Prefix Tree) = autocomplete & prefix search
 - Positional Index = stores word positions (phrase search)
 - Compressed Bitmaps = space efficient storage
- (vi) Process

- Tokenization
- Stop word removal: remove common words
- Stemming/Lemmatization: reduce to root form
- Metadata extraction: title, author, URL, etc
- Build inverted index: term $\rightarrow$ documents

(v) Storage & Optimization

- Stored in distributed systems
- Uses compression techniques
- Stores ranking signals

(vi) Challenges

- Handling large scale data
- Real time index updates
- Minimizing latency & refresh time

Query Processing -

- (i) Tokenization: break query into words
- (ii) Spelling correction: fix errors & suggest queries
- (iii) Query expansion: add synonyms/related
- (iv) Intent understanding: use NLP to detect user goal

Ranking -

- (i) TF-IDF: Term importance (freq + rarity)
- (ii) BM25: Improved probabilistic ranking
- (iii) ML models: learn ranking
- (iv) Deep learning: semantic understanding

Ranking Signals -

- (i) On-page: keywords, title, content
- (ii) Off-page: backlinks, authority
- (iii) Technical: speed, mobile friendliness
- (iv) User signals: CTR, dwell time
- (v) Personalization: location, history

*** Key Idea = combine signals + algorithms $\rightarrow$ ranked results**

Scalability -

- (i) Horizontal scaling
 - (ii) Sharding
 - (iii) Replication
 - (iv) Load Balancing
- Unit-1

Performance optimization -

- (i) Caching
- (ii) In memory caches: faster access
- (iii) Pagination: handle large results efficiently

Key metrics -

- (i) Latency
- (ii) Throughput
- (iii) Index size
- (iv) Cache hit ratio

Modern Trends in Search Engines -

- (i) AI/LLM integration = direct conversational answers
- (ii) Semantic search = focus on meaning & user intent
- (iii) Multimodal search = search across text, images, videos, audio
- (iv) Advanced trends
 - Real time indexing: faster updates for dynamic content
 - Personalized ranking: results based on user context/history
- (v) Key challenges
 - Scalability
 - Energy impact
 - Misinformation control
 - Privacy preservation

Search Engine Ranking (Page Rank) -

- (i) Pages modeled as a directed graph (nodes = pages, edges = links)
- (ii) Backlinks = importance signals (more / quality links $\rightarrow$ higher rank)
- (iii) Links can have weights (important pages give stronger votes)

(iv) Core Idea:

- Rank depends on:
 - No. of incoming links
 - Quality of linking pages
- Not all links are equal $\rightarrow$ authority matters

(v) PageRank is a probability distribution

- (vi) Final rank represent likelihood of visiting a page
- (vii) Used as a ranking signal

(viii) Problem Solved:

- Determines page is most important/relevant among many
- Based on link structure (backlinks)

(ix) Model:

- Web = Graph + links
- A page with more incoming edges $\rightarrow$ higher importance

(x) PageRank calculation:

- Start with initial equal probability for all pages
 $\rightarrow PR = 1/N$
- Update ranks iteratively using link structure
- Continue until steady state (no major change)

Recommendation Systems -

- (i) Sophisticated information filtering tools that predict what users will like based on their past behavior, preferences
- (ii) Goal: to guide users to relevant products, content, or services in an increasingly vast digital landscape
- (iii) Enhances user experience, driving engagement & boosting business metrics

Content-Based Filtering -

- (i) Recommends items similar to user's past interests
- (ii) Uses item features + user profiles
- (iii) Builds profile from liked/rated items
- (iv) How:

Used:

- Distributed systems
- No SQL DBs
- Caching
- ML models

(v) Components:

- Item Features: Metadata
- User Profile: Feature vector of user preferences
- Similarity Measure: Cosine/Euclidean/vector based

(vi) Advantages:

- Cold start for users
- Explainable recommendations
- No user sparsity

(vii) Disadvantages:

- Limited diversity
- Requires good feature extraction
- Cold start for items

Collaborative Filtering - (User-Based)

- (i) Recommends using similar user's preference
- (ii) Assumption: users with similar tastes $\rightarrow$ similar future choices

Identify Similar Users -

- (i) Find users with similar ratings/interactions
- (ii) Use similarity measures:
 - Cosine similarity
 - Pearson correlation

Aggregate Preferences -

- (i) Select nearest neighbors (similar users)
- (ii) Find items they liked but target user hasn't seen

Generate Recommendations -

- (i) Predict = weighted (based on rating average similarity)
- (ii) Recommend items with highest predicted ratings

Advantages -

- (i) Discover serendipitous items
- (ii) No feature engineering needed
- (iii) Dynamic

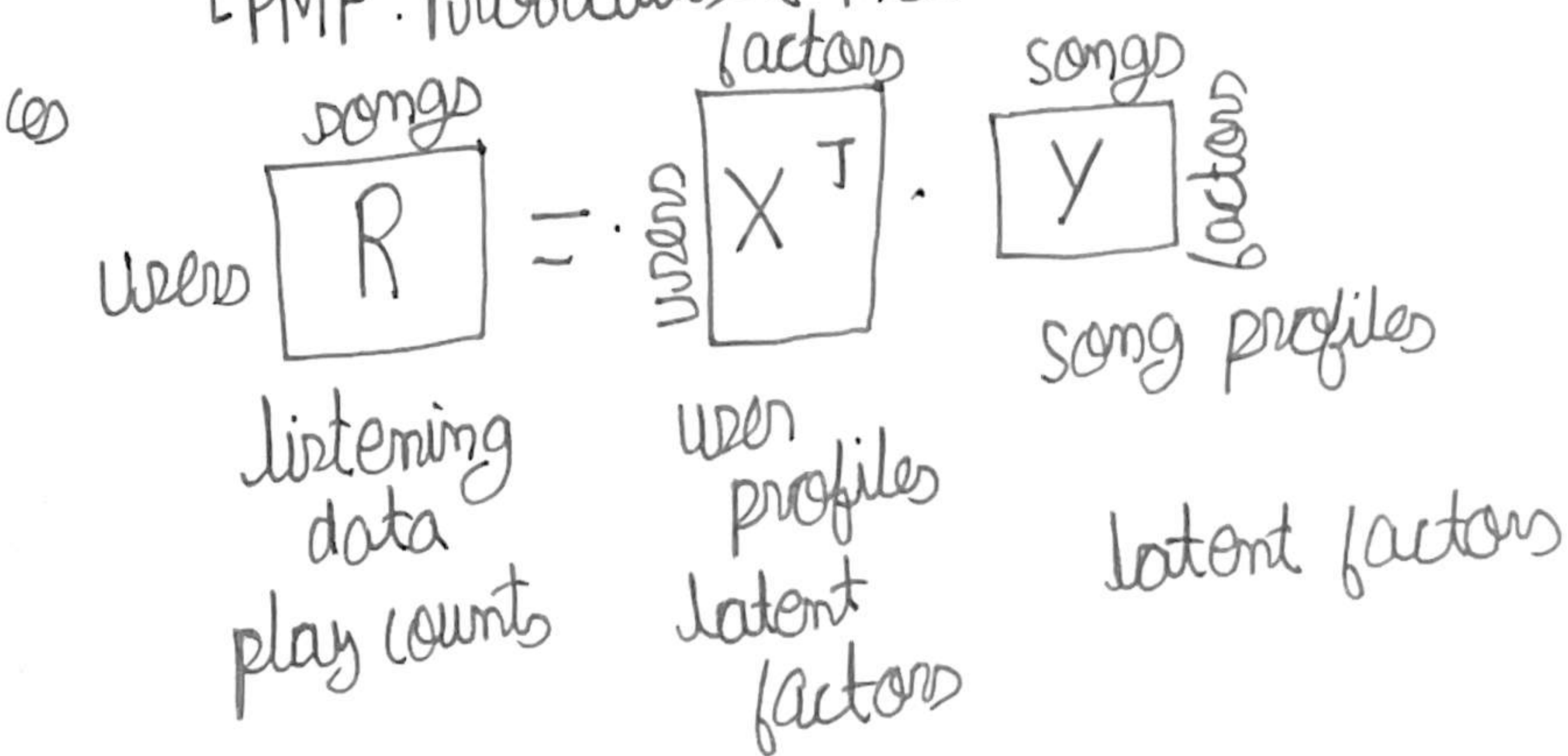
Disadvantages -

- (i) Cold Start Problem
- (ii) User Sparsity
- (iii) Computational Cost
- (iv) Scalability issues

Matrix Factorization & Latent Factors -

- (i) Decomposes user item matrix $\rightarrow$ low dimensional matrices
- (ii) Learns hidden (latent) features of users & items
- (iii) Captures implicit patterns beyond similarity
- (iv) Common techniques:

- SVD: Matrix decomposition (optimized for sparse data)
- NMF: Non negative factors $\rightarrow$ more interpretable
- PMF: Probabilistic model with noise handling



Hybrid Recommendation Systems -

- (i) Combined Collaborative + content based
- (ii) Solves

- Cold Start
 - Sparsity
 - Low diversity
 - Improved accuracy
- } Benefits

- (iii) Methods

- Weighted: combine scores $(\alpha \cdot CF + (1-\alpha) \cdot \text{Content})$
- Cascading: One model filters, another ranks
- Feature combination: merge features $\rightarrow$ single model
- Meta level: output of one $\rightarrow$ input of another

Evaluation & Metrics -

- (i) Evaluate effectiveness of recommender systems
- (ii) Uses multiple metrics (accuracy + diversity + novelty)

Accuracy metrics -

- (i) RMSE: measures error magnitude (sensitive to outliers)
- (ii) MAE: average error (less sensitive to outliers)

Ranking metrics -

- (i) Precision @ K: Relevant items in top K
- (ii) Recall @ K: Coverage of relevant items in top K
- (iii) NDCG: Considers ranking position (top items matter more)

Beyond Accuracy -

- (i) Coverage: % of items/users recommended
- (ii) Diversity: Variety in recommendations
- (iii) Novelty: New/unseen items
- (iv) Serendipity: Unexpected but useful items

A/B Testing -

- (i) Compares models on real users
- (ii) Metrics:
 - CTR (Click through rate)
 - Conversion rate
 - Time spent
 - Revenue

Future Directions -

- (i) Multi modal: Text, image, audio integration
- (ii) Causal inference: Understand cause-effect
- (iii) Fairness & Reduce bias Ethics
- (iv) Explainable AI (XAI): Transparent recommendations

Real World Applications -

- (i) Netflix: Hybrid (CF + content + deep learning + context)
- (ii) Amazon: Item based CF + content + user behavior
- (iii) Spotify: CF + content (audios) + deep learning

Cost Estimation -

- (i) Predicts cost of building & running systems
- (ii) Helps in budgets & resource allocation
- (iii) More complex in cloud/distributed systems
- (iv) Importance
 - Financial planning
 - Avoid unexpected costs
 - Optimize resource usage
 - Support system design decisions

#Cloud Computing -

- (i) On demand resource via internet
- (ii) No need for physical infrastructure
- (iii) Scalable & elastic

#Pay-As-You-Go Model -

- (i) Pay only for used resources
- (ii) No upfront cost
- (iii) Suitable for startups & dynamic workloads

#Resource-Based Pricing -

- (i) Cost based on:
 - CPU
 - Memory
 - Storage
- (ii) Extra cost for data transfer

#Spot & Reserved Instances -

- (i) Spot: Cheap, can be terminated anytime
- (ii) Reserved: long term, discounted pricing

#Distributed vs Cloud Cost -

- (i) Distributed: High upfront + maintenance systems
- (ii) Cloud: Flexible, usage-based
- (iii) Cloud reduces infra & maintenance burden

#Cloud Cost Models -

- (i) Usage based pricing (pay for what you use)
- (ii) Flexible & cost efficient

#Subscription Model -

- (i) Fixed periodic payment
- (ii) Limited resources per plan
- (iii) Predictable cost

#Tiered Pricing -

- (i) Usage divided into tiers/steps
- (ii) Higher usage $\rightarrow$ lower per unit cost
- (iii) Good for large scale systems

Tradeoffs -

Key Quality attributes -

- (i) Performance
- (ii) Scalability
- (iii) Reliability
- (iv) Security
- (v) Maintainability
- (vi) Cost

Performance vs Scalability -

Performance

- Measured response speed (latency ↓, throughput ↑)

Scalability

- Handles increasing load by adding resources
- Focus on growth over time

Performance vs Cost -

Performance

- Low response time
- High throughput
- Efficient resource usage

Cost

- Hardware
- Infrastructure
- Maintenance & operations

* TRADEOFFS -

- (a) High Performance → tight coupling
↳ ↓ scalability
- (b) High Scalability → distributed systems → ↑ latency
- (c) eg
 - Single server → fast, not scalable
 - Micro services → scalable, slightly slower

* TRADEOFFS -

- (a) ↑ Performance ⇒ ↑ Cost
- (b) ↓ Cost ⇒ ↓ Performance
- (c) eg
 - High end server → expensive, fast
 - Shared hosting → cheap, slower

Scalability vs Consistency -

Scalability

- Handle more users via distribution & horizontal scaling

Consistency

- All users see same/latest data
- Requires synchronization

* TRADEOFFS -

- (a) Strong consistency → ↓ scalability, ↑ latency
- (b) High scalability → relaxed consistency (eventual)

Availability vs Consistency -

Consistency

- Same data for all users at all times

Availability

- System always responds, even during failures

Why Trade-off exists -

- Network delay
- Distributed systems complexity
- Data replication across nodes
- Explained by CAP theorem

Reliability vs Cost -

Reliability

└ System works even during failures

Cost

└ Infrastructure + maintenance expense

* TRADEOFFS -

↑ Reliability $\Rightarrow$ Replication, backups, failover $\Rightarrow$ ↑ Cost

Eg

└ Single server $\rightarrow$ Low cost, low reliability

└ Multi region $\rightarrow$ High cost, high reliability

Fault Tolerance vs Performance -

Fault Tolerance

└ Continued operation despite failures

└ Uses replication, backup, failover

Performance

└ Speed & efficiency

* TRADEOFFS -

(a) Fault tolerance adds
└ Extra communication
└ Data replication
└ Consistency checks

(b) ↑ Reliability $\Rightarrow$ ↓ Performance

Security vs Usability -

Security

└ Protects from unauthorized access

Usability

└ Ease of use

* TRADEOFFS -

(a) Strong security
└ MFA, frequent authentication
└ ↓ Usability

(b) eg

└ Banking apps $\rightarrow$ high security, lower usability

└ Public sites $\rightarrow$ high usability, lower security

Maintainability vs Performance -

Maintainability

└ Easy to update, debug, modify

Performance

└ Execution speed

* TRADEOFFS -

(a) Modular/clean code $\rightarrow$ easy maintenance, slower

(b) Highly optimized code $\rightarrow$ fast, hard to maintain

(c) eg

└ Enterprise systems $\rightarrow$ maintainability preferred

└ Real time systems $\rightarrow$ performance preferred

Replication vs Storage Cost -

Replication

- Multiple copies of data
- Improves availability & fault tolerance

Storage Cost

- Extra disks + backup + maintenance cost

* TRADEOFFS -

- (a) More replication $\Rightarrow$ $\uparrow$ cost, $\uparrow$ reliability
- (b) Less replication $\Rightarrow$ $\downarrow$ cost, $\downarrow$ reliability
- (c) eg
 - Banking systems $\rightarrow$ high replication
 - Log systems $\rightarrow$ low replication

Centralized vs Distributed systems -

Centralized = easier & cheaper but less reliable

Distributed = scalable & fault tolerant but complex & costly

Monolithic vs Microservices (Unit-1)

Latency vs Throughput (Unit-1)

Automation vs Control -

Automation

- Operations handled automatically
- Pros: fast, consistent, scalable, less human error
- Cons: less flexibility, harder debugging, hidden internals

Control

- Humans manage & monitor
- Pros: flexible, handles edge case, better visibility
- Cons: slower, human errors, not scalable

Cloud vs On-Premise

Aspect

1) Cost Model

Cloud

Pay as you go (OPEX)

2) Deployment

Third party hosted

3) Scalability

Highly scalable

4) Control

Less control over infra

5) Maintenance

Handled by provider

6) Security

Shared responsibility

7) Flexibility

High

8) Deployment Speed

Fast

On-Premise

High upfront cost (CAPEX)

In-house/local setup

Limited scalability

Full control over system

Managed by organization

Higher control

Less

Slow

#System Optimization -

- Improves performance, efficiency,
- Better use of hardware, software, network resources
- Essential for scalable & distributed systems

Why?

- ↓ Response time, ↑ throughput
- ↓ infrastructure & ~~operational~~ operational cost
- Improves user experience
- Ensures stability under load

Areas

- Performance (speed, latency)
- Scalability (handle more users/load)
- Resource Utilization (CPU, memory)
- Reliability of ~~optimization~~ (fault tolerance, uptime)
- Cost (reduce infra & operational expenses)

#Performance Optimization

Techniques

- Caching
- Efficient algorithms
- Asynchronous processing
- Reducing I/O operations

Benefits

- Faster response time
- Better user experience

Trade off

- Increased memory usage
- Complexity in cache invalidation

#Scalability Optimization

Techniques

- Horizontal scaling
- Vertical scaling
- Stateless services
- Auto scaling mechanisms

Benefits

- Handles increased load
- Supports growth

Trade off

- Data consistency challenges
- High infra cost

Resource Utilization Optimization - Techniques

- Dynamic Resource Allocation
 - Utilization
 - Resources assigned only when needed
 - Cost efficient
- Containerization
 - Lightweight
 - Faster than VM
 - Less memory usage
- Scheduling
 - CPU scheduling
 - Job scheduling

Reliability Optimization Techniques

- Replication
- Failover mechanism (switch to backup system)
- Graceful degradation (continued with reduced functionality)

Cost Optimization - Techniques

- Auto scaling (~~avoid~~ avoids over provisioning)
- Reserved & Spot instances
 - Reserved (cheaper for long term use)
 - Spot (very cheap but unreliable)
- Storage Tiering
 - Hot data → expensive storage
 - Cold data → cheap storage
 - Optimized cost

Data Optimization - Techniques

- Indexing (creates paths to databases)
- Data Partitioning (sharding)
- Data compression

Tradeoffs -

- (i) Performance vs Cost
- (ii) Scalability vs Consistency
- (iii) Reliability vs Performance
- (iv) Optimization depends on goal

Best practices -

- Identify system bottlenecks
- Monitor performance
- Optimize incrementally
- Document optimization techniques